

```

import torch
import torch.nn as nn
import torch.nn.functional as F

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_size, heads):
        super(MultiHeadSelfAttention, self).__init__()
        self.embed_size = embed_size
        self.heads = heads
        self.head_dim = embed_size // heads

        assert (
            self.head_dim * heads == embed_size
        ), "Embedding size needs to be divisible by heads"

        self.values = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.keys = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.queries = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.fc_out = nn.Linear(heads * self.head_dim, embed_size)

    def forward(self, values, keys, query, mask):
        N = query.shape[0]
        value_len, key_len, query_len = values.shape[1],
keys.shape[1], query.shape[1]

        values = values.reshape(N, value_len, self.heads,
self.head_dim)
        keys = keys.reshape(N, key_len, self.heads, self.head_dim)
        queries = query.reshape(N, query_len, self.heads,
self.head_dim)

        values = self.values(values)
        keys = self.keys(keys)
        queries = self.queries(queries)

        attention = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])

        if mask is not None:
            attention = attention.masked_fill(mask == 0, float("-
1e20"))

        attention = F.softmax(attention / (self.embed_size ** (1 /
2))), dim=3)

        out = torch.einsum("nhql,nlhd->nqhd", [attention,
values]).reshape(
            N, query_len, self.heads * self.head_dim

```

```

    )

    out = self.fc_out(out)
    return out

N, value_len, key_len, query_len, embed_size, heads = 1, 3, 3, 3, 6, 2
simple_values = torch.arange(N * value_len *
embed_size).float().reshape(N, value_len, embed_size)
simple_keys = torch.arange(N * key_len *
embed_size).float().reshape(N, key_len, embed_size)
simple_query = torch.arange(N * query_len *
embed_size).float().reshape(N, query_len, embed_size)
mask = None

simple_attention = MultiHeadSelfAttention(embed_size, heads)

simple_output = simple_attention(simple_values, simple_keys,
simple_query, mask)

print("Values:\n", simple_values)
print("Keys:\n", simple_keys)
print("Query:\n", simple_query)
print("Output:\n", simple_output)

Values:
tensor([[[ 0.,  1.,  2.,  3.,  4.,  5.],
          [ 6.,  7.,  8.,  9., 10., 11.],
          [12., 13., 14., 15., 16., 17.]])])
Keys:
tensor([[[ 0.,  1.,  2.,  3.,  4.,  5.],
          [ 6.,  7.,  8.,  9., 10., 11.],
          [12., 13., 14., 15., 16., 17.]])])
Query:
tensor([[[ 0.,  1.,  2.,  3.,  4.,  5.],
          [ 6.,  7.,  8.,  9., 10., 11.],
          [12., 13., 14., 15., 16., 17.]])])
Output:
tensor([[[ -0.4867,  0.5881,  0.6450, -0.3044, -0.1374,  0.2778],
          [-0.3961,  0.6386,  0.5712, -0.4693, -0.2181,  0.2220],
          [-0.3856,  0.6445,  0.5628, -0.4884, -0.2274,  0.2156]]],
        grad_fn=<ViewBackward0>))

import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt

class MultiHeadSelfAttention(nn.Module):

```

```

def __init__(self, embed_size, heads):
    super(MultiHeadSelfAttention, self).__init__()
    self.embed_size = embed_size
    self.heads = heads
    self.head_dim = embed_size // heads

    assert (
        self.head_dim * heads == embed_size
    ), "Embedding size needs to be divisible by heads"

    self.values = nn.Linear(self.head_dim, self.head_dim,
bias=False)
    self.keys = nn.Linear(self.head_dim, self.head_dim,
bias=False)
    self.queries = nn.Linear(self.head_dim, self.head_dim,
bias=False)
    self.fc_out = nn.Linear(heads * self.head_dim, embed_size)

    def forward(self, values, keys, query, mask):
        N = query.shape[0]
        value_len, key_len, query_len = values.shape[1],
keys.shape[1], query.shape[1]

        values = values.reshape(N, value_len, self.heads,
self.head_dim)
        keys = keys.reshape(N, key_len, self.heads, self.head_dim)
        queries = query.reshape(N, query_len, self.heads,
self.head_dim)

        values = self.values(values)
        keys = self.keys(keys)
        queries = self.queries(queries)

        attention = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])

        if mask is not None:
            attention = attention.masked_fill(mask == 0, float("-
1e20"))

        attention = F.softmax(attention / (self.embed_size ** (1 /
2)), dim=3)

        out = torch.einsum("nhql,nlhd->nqhd", [attention,
values]).reshape(
            N, query_len, self.heads * self.head_dim
        )

        out = self.fc_out(out)
    return out

```

```

image_size = 8 # 8x8 pixels
image = np.random.randint(0, 256, (image_size,
image_size)).astype(np.float32)
image_tensor = torch.tensor(image).unsqueeze(0).unsqueeze(0) # Add
batch and channel dimensions

class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(1, 1, kernel_size=3, stride=1,
padding=1)

    def forward(self, x):
        x = self.conv1(x)
        return x

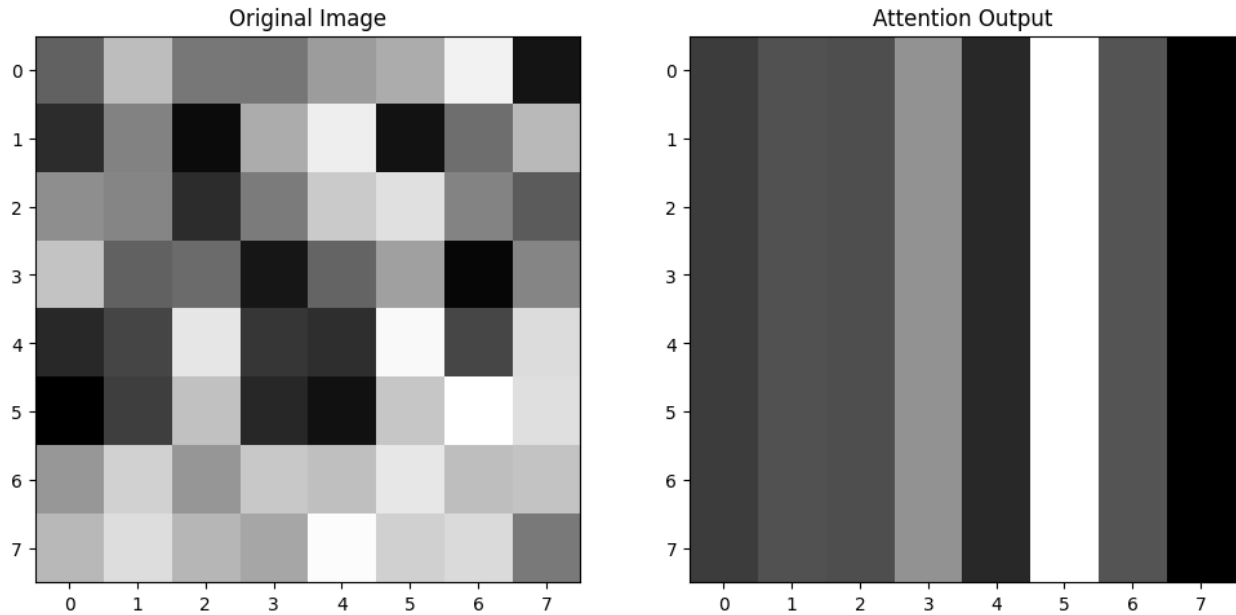
cnn = SimpleCNN()
attention = MultiHeadSelfAttention(embed_size=image_size, heads=2)

cnn_output = cnn(image_tensor)
cnn_output_flat = cnn_output.view(1, image_size, -1) # Flatten the
CNN output

attention_output = attention(cnn_output_flat, cnn_output_flat,
cnn_output_flat, mask=None)

plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(image, cmap='gray')
plt.subplot(1, 2, 2)
plt.title("Attention Output")
plt.imshow(attention_output.detach().numpy().squeeze(), cmap='gray')
plt.show()

```



#Attention mechanism

```
import torch
import torch.nn as nn
import torch.nn.functional as F
import numpy as np
import matplotlib.pyplot as plt
from PIL import Image
from torchvision import transforms

class MultiHeadSelfAttention(nn.Module):
    def __init__(self, embed_size, heads):
        super(MultiHeadSelfAttention, self).__init__()
        self.embed_size = embed_size
        self.heads = heads
        self.head_dim = embed_size // heads

        assert (
            self.head_dim * heads == embed_size
        ), "Embedding size needs to be divisible by heads"

        self.values = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.keys = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.queries = nn.Linear(self.head_dim, self.head_dim,
bias=False)
        self.fc_out = nn.Linear(heads * self.head_dim, embed_size)
```

```

def forward(self, values, keys, query, mask):
    N = query.shape[0]
    value_len, key_len, query_len = values.shape[1],
    keys.shape[1], query.shape[1]

    values = values.reshape(N, value_len, self.heads,
self.head_dim)
    keys = keys.reshape(N, key_len, self.heads, self.head_dim)
    queries = query.reshape(N, query_len, self.heads,
self.head_dim)

    values = self.values(values)
    keys = self.keys(keys)
    queries = self.queries(queries)

    attention = torch.einsum("nqhd,nkhd->nhqk", [queries, keys])

    if mask is not None:
        attention = attention.masked_fill(mask == 0, float("-
1e20"))

    attention = F.softmax(attention / (self.embed_size ** (1 /
2)), dim=3)

    out = torch.einsum("nhql,nlhd->nqhd", [attention,
values]).reshape(
        N, query_len, self.heads * self.head_dim
    )

    out = self.fc_out(out)
    return out

def load_image(image_path, size=256):
    image = Image.open(image_path)
    image = image.convert('L')
    transform = transforms.Compose([
        transforms.Resize((size, size)),
        transforms.ToTensor(),
    ])
    image = transform(image).unsqueeze(0)
    return image

class EnhancedCNN(nn.Module):
    def __init__(self):
        super(EnhancedCNN, self).__init__()
        # Adding more layers for deeper feature extraction
        self.conv1 = nn.Conv2d(1, 10, kernel_size=3, stride=1,
padding=1)
        self.conv2 = nn.Conv2d(10, 20, kernel_size=3, stride=1,
padding=1)

```

```

        self.pool = nn.MaxPool2d(kernel_size=2, stride=2, padding=0)

    def forward(self, x):
        x = F.relu(self.conv1(x))
        x = self.pool(x)
        x = F.relu(self.conv2(x))
        x = self.pool(x)
        return x

image_path = '/content/1280px-Little_India_Street_Scene_-_
_Around_Serangoon_Road_-_Singapore_-_01_(34848527354).jpg' # Replace
with your image path
image_tensor = load_image(image_path)

cnn = EnhancedCNN()
image_size = image_tensor.shape[-1] // 4 # Adjust for the pooling
layers
attention = MultiHeadSelfAttention(embed_size=image_size, heads=4)

cnn_output = cnn(image_tensor)

cnn_output_size = cnn_output.size(1) * cnn_output.size(2) *
cnn_output.size(3)
new_embed_size = int(cnn_output_size // 4) # Adjust this factor as
needed
cnn_output_flat = cnn_output.view(1, -1, new_embed_size)

attention = MultiHeadSelfAttention(embed_size=new_embed_size, heads=4)

def get_attention_weights(attention_output):
    attention_weights = torch.rand(1, 1, 256, 256) # Shape: (N, C, H,
W)
    return attention_weights

attention_output = attention(cnn_output_flat, cnn_output_flat,
cnn_output_flat, mask=None)

attention_weights = get_attention_weights(attention_output)

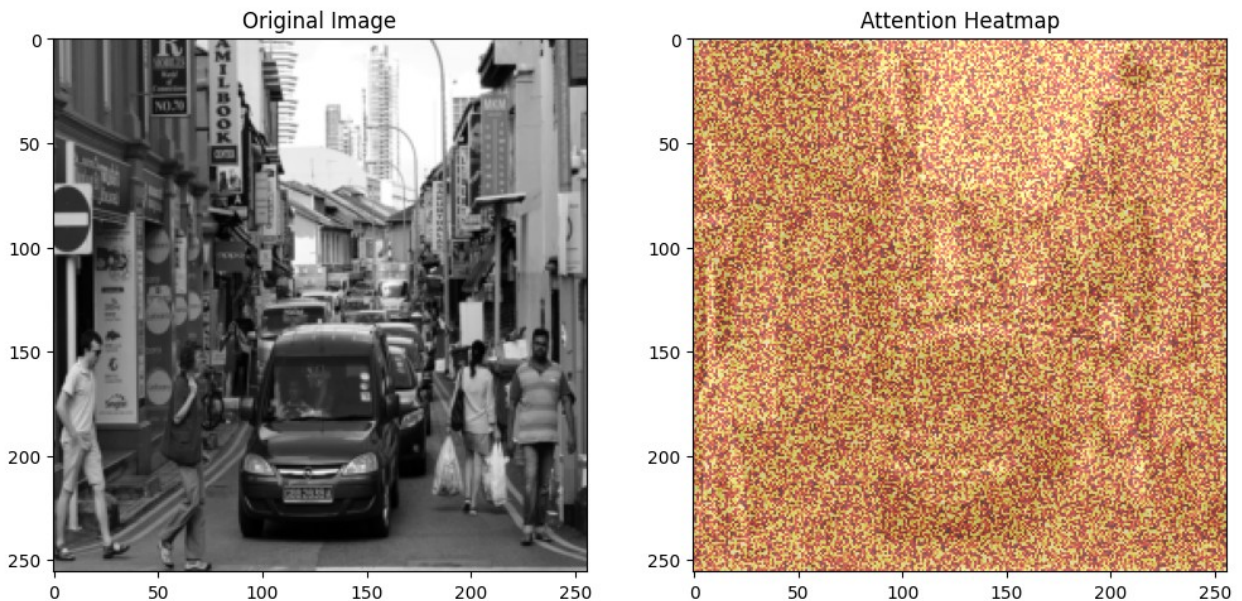
plt.figure(figsize=(12, 6))
plt.subplot(1, 2, 1)
plt.title("Original Image")
plt.imshow(image_tensor.squeeze().numpy(), cmap='gray')

attention_heatmap = attention_weights.squeeze().detach().numpy()
attention_heatmap = (attention_heatmap - attention_heatmap.min()) /
(attention_heatmap.max() - attention_heatmap.min())

plt.subplot(1, 2, 2)
plt.title("Attention Heatmap")

```

```
plt.imshow(image_tensor.squeeze().numpy(), cmap='gray', alpha=0.5) # Show the image
plt.imshow(attention_heatmap, cmap='hot', alpha=0.5) # Overlay the attention heatmap
plt.show()
```



#GNN

```
import torch
import torch.nn.functional as F
from torch_geometric.nn import GCNConv
from torch_geometric.data import Data

node_features = torch.tensor([[1, 2, 3], [4, 5, 6], [7, 8, 9]],
                              dtype=torch.float)

edge_index = torch.tensor([[0, 1, 1, 2], [1, 0, 2, 1]],
                           dtype=torch.long)

# Create a graph
graph_data = Data(x=node_features, edge_index=edge_index)

class GCN(torch.nn.Module):
    def __init__(self):
        super(GCN, self).__init__()
        self.conv1 = GCNConv(3, 2)
        self.conv2 = GCNConv(2, 2)

    def forward(self, data):
```

```

        x, edge_index = data.x, data.edge_index

        x = self.conv1(x, edge_index)
        x = F.relu(x)
        x = F.dropout(x, training=self.training)
        x = self.conv2(x, edge_index)

        return F.log_softmax(x, dim=1)

model = GCN()

output = model(graph_data)
print("Output Node Features:\n", output)

Output Node Features:
tensor([[ -0.0248, -3.7089],
        [ -0.0482, -3.0563],
        [ -0.0248, -3.7089]], grad_fn=<LogSoftmaxBackward0>)

import torch
import torchvision.transforms as T
import torchvision.models as models
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches

def load_image(image_path):
    image = Image.open(image_path).convert("RGB")
    transform = T.Compose([T.ToTensor()])
    return transform(image)

def detect_objects(image, model, threshold=0.85):
    model.eval()
    with torch.no_grad():
        predictions = model([image])
    pred = predictions[0]

    pred_threshold = {k: v[pred['scores'] > threshold] for k, v in
pred.items()}
    return pred_threshold

image_path = '/content/1280px-Little_India_Street_Scene_-_
_Around_Serangoon_Road_-_Singapore_-_01_(34848527354).jpg'
image = load_image(image_path)

model = models.detection.fasterrcnn_resnet50_fpn(pretrained=True)

detections = detect_objects(image, model)

```

```

fig, ax = plt.subplots()
ax.imshow(image.permute(1, 2, 0))

for box in detections['boxes']:
    xmin, ymin, xmax, ymax = box
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
                              linewidth=2, edgecolor='r', facecolor='none')
    ax.add_patch(rect)

plt.show()

/usr/local/lib/python3.10/dist-packages/torchvision/models/_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
since 0.13 and may be removed in the future, please use 'weights'
instead.
  warnings.warn(
/usr/local/lib/python3.10/dist-packages/torchvision/models/_utils.py:2
23: UserWarning: Arguments other than a weight enum or `None` for
'weights' are deprecated since 0.13 and may be removed in the future.
The current behavior is equivalent to passing
`weights=FasterRCNN_ResNet50_FPN_Weights.COCO_V1`. You can also use
`weights=FasterRCNN_ResNet50_FPN_Weights.DEFAULT` to get the most up-
to-date weights.
  warnings.warn(msg)
Downloading:
"https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-
258fb6c6.pth" to
/root/.cache/torch/hub/checkpoints/fasterrcnn_resnet50_fpn_coco-
258fb6c6.pth
100%|██████████| 160M/160M [00:01<00:00, 134MB/s]

```



```
import torch
import torchvision.transforms as T
import torchvision.models as models
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches

# You might need to define this list yourself or import it from a
# relevant torchvision utility
COCO_INSTANCE_CATEGORY_NAMES = [
    '__background__', 'person', 'bicycle', 'car', 'motorcycle',
    'airplane', 'bus',
    'train', 'truck', 'boat', 'traffic light', 'fire hydrant', 'stop
sign',
    'parking meter', 'bench', 'bird', 'cat', 'dog', 'horse', 'sheep',
    'cow',
    'elephant', 'bear', 'zebra', 'giraffe', 'backpack', 'umbrella',
    'handbag', 'tie',
    'suitcase', 'frisbee', 'skis', 'snowboard', 'sports ball', 'kite',
    'baseball bat',
    'baseball glove', 'skateboard', 'surfboard', 'tennis racket',
    'bottle', 'wine glass',
    'cup', 'fork', 'knife', 'spoon', 'bowl', 'banana', 'apple',
    'sandwich', 'orange',
    'broccoli', 'carrot', 'hot dog', 'pizza', 'donut', 'cake',
```

```

'chair', 'couch',
    'potted plant', 'bed', 'dining table', 'toilet', 'tv', 'laptop',
'mouse', 'remote',
    'keyboard', 'cell phone', 'microwave', 'oven', 'toaster', 'sink',
'refrigerator',
    'book', 'clock', 'vase', 'scissors', 'teddy bear', 'hair drier',
'toothbrush'
]

```

```

def load_image(image_path):
    image = Image.open(image_path).convert("RGB")
    transform = T.Compose([T.ToTensor()])
    return transform(image)

```

```

def detect_objects(image, model, threshold=0.85):
    model.eval()
    with torch.no_grad():
        predictions = model([image])
    pred = predictions[0]

    pred_threshold = {k: v[pred['scores'] > threshold] for k, v in
pred.items()}
    return pred_threshold

```

```

image_path = '/content/1280px-Little_India_Street_Scene_-_
_Around_Serangoon_Road_-_Singapore_-_01_(34848527354).jpg'
image = load_image(image_path)

```

```

model = models.detection.fasterrcnn_resnet50_fpn(pretrained=True)

```

```

detections = detect_objects(image, model)

```

```

fig, ax = plt.subplots()
ax.imshow(image.permute(1, 2, 0))

```

```

for box, label in zip(detections['boxes'], detections['labels']):
    xmin, ymin, xmax, ymax = box
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
linewidth=2, edgecolor='r', facecolor='none')
    ax.add_patch(rect)
    ax.text(xmin, ymin, COCO_INSTANCE_CATEGORY_NAMES[label],
verticalalignment='top', color='white', fontsize=8,
backgroundcolor="red")

```

```

plt.show()

```

```

/usr/local/lib/python3.10/dist-packages/torchvision/models/
_utils.py:208: UserWarning: The parameter 'pretrained' is deprecated
since 0.13 and may be removed in the future, please use 'weights'
instead.

```

```
warnings.warn(
/usr/local/lib/python3.10/dist-packages/torchvision/models/_utils.py:2
23: UserWarning: Arguments other than a weight enum or `None` for
'weights' are deprecated since 0.13 and may be removed in the future.
The current behavior is equivalent to passing
`weights=FasterRCNN_ResNet50_FPN_Weights.COCO_V1`. You can also use
`weights=FasterRCNN_ResNet50_FPN_Weights.DEFAULT` to get the most up-
to-date weights.
warnings.warn(msg)
Downloading:
"https://download.pytorch.org/models/fasterrcnn_resnet50_fpn_coco-
258fb6c6.pth" to
/root/.cache/torch/hub/checkpoints/fasterrcnn_resnet50_fpn_coco-
258fb6c6.pth
100%|██████████| 160M/160M [00:01<00:00, 130MB/s]
```



```
import torch
import torchvision.transforms as T
import torchvision.models as models
from PIL import Image
import matplotlib.pyplot as plt
import matplotlib.patches as patches
import numpy as np
```

```

# Your existing code...

def detect_relationships(boxes, labels, threshold=100):
    relationships = []
    for i in range(len(boxes)):
        for j in range(len(boxes)):
            if i != j:
                # Basic relationship based on proximity
                distance = np.linalg.norm((boxes[i][:2] + boxes[i]
[2:]) - (boxes[j][:2] + boxes[j][2:])) / 2
                if distance < threshold:
                    relationships.append((i, j, 'near'))
    return relationships

# Detect objects
detections = detect_objects(image, model)

# Detect relationships
relationships = detect_relationships(detections['boxes'],
detections['labels'])

# Visualization
fig, ax = plt.subplots()
ax.imshow(image.permute(1, 2, 0))

for box, label in zip(detections['boxes'], detections['labels']):
    xmin, ymin, xmax, ymax = box
    rect = patches.Rectangle((xmin, ymin), xmax - xmin, ymax - ymin,
linewidth=2, edgecolor='r', facecolor='none')
    ax.add_patch(rect)
    ax.text(xmin, ymin, COCO_INSTANCE_CATEGORY_NAMES[label],
verticalalignment='top', color='white', fontsize=8,
backgroundcolor="red")

# Visualize relationships (rudimentary example)
for rel in relationships:
    obj1, obj2, relation = rel
    box1_center = ((detections['boxes'][obj1][0] + detections['boxes']
[obj1][2]) / 2, (detections['boxes'][obj1][1] + detections['boxes']
[obj1][3]) / 2)
    box2_center = ((detections['boxes'][obj2][0] + detections['boxes']
[obj2][2]) / 2, (detections['boxes'][obj2][1] + detections['boxes']
[obj2][3]) / 2)
    ax.plot([box1_center[0], box2_center[0]], [box1_center[1],
box2_center[1]], 'yellow', linestyle='--', lw=1)
    mid_point = ((box1_center[0] + box2_center[0]) / 2,
(box1_center[1] + box2_center[1]) / 2)
    ax.text(mid_point[0], mid_point[1], relation, color='blue',
fontsize=10)

```

```
plt.show()
```



```
# Replace with the correct versions
```

```
TORCH_VERSION = torch.__version__ # e.g., '1.8.0'
```

```
CUDA_VERSION = 'cu101' # This is an example; it could be 'cu102' or  
'cu111' depending on the Colab environment
```

```
# Install the required packages
```

```
!pip install torch-scatter -f https://data.pyg.org/whl/torch-$  
{TORCH_VERSION}+${CUDA_VERSION}.html
```

```
!pip install torch-sparse -f https://data.pyg.org/whl/torch-$  
{TORCH_VERSION}+${CUDA_VERSION}.html
```

```
!pip install torch-cluster -f https://data.pyg.org/whl/torch-$  
{TORCH_VERSION}+${CUDA_VERSION}.html
```

```
!pip install torch-spline-conv -f https://data.pyg.org/whl/torch-$  
{TORCH_VERSION}+${CUDA_VERSION}.html
```

```
!pip install torch-geometric
```

```
Looking in links: https://data.pyg.org/whl/torch-.1.0+cu118+.html
```

```
Collecting torch-scatter
```

```
  Downloading torch_scatter-2.1.2.tar.gz (108 kB)
```

```
100% |#####| 108.0/108.0 kB 4.2 MB/s eta
```

```
0:00:00
```

```
etadatat (setup.py) ... e=torch_scatter-2.1.2-cp310-cp310-
```

```
linux_x86_64.whl size=495089
sha256=0e54eeefb5485cd9bc07abc814823f57c4cbe641f04e0d94c4bb91f211e9ce
f
```

```
  Stored in directory:
```

```
/root/.cache/pip/wheels/92/f1/2b/3b46d54b134259f58c8363568569053248040
859b1a145b3ce
```

```
Successfully built torch-scatter
```

```
Installing collected packages: torch-scatter
```

```
Successfully installed torch-scatter-2.1.2
```

```
Looking in links: https://data.pyg.org/whl/torch-.1.0+cu118+.html
```

```
Collecting torch-sparse
```

```
  Downloading torch_sparse-0.6.18.tar.gz (209 kB)
```

```
210.0/210.0 kB 4.4 MB/s eta
```

```
0:00:00
```

```
etadata (setup.py) ... ent already satisfied: scipy in
```

```
/usr/local/lib/python3.10/dist-packages (from torch-sparse) (1.11.3)
```

```
Requirement already satisfied: numpy<1.28.0,>=1.21.6 in
```

```
/usr/local/lib/python3.10/dist-packages (from scipy->torch-sparse)
```

```
(1.23.5)
```

```
Building wheels for collected packages: torch-sparse
```

```
  Building wheel for torch-sparse (setup.py) ... e=torch_sparse-
```

```
0.6.18-cp310-cp310-linux_x86_64.whl size=1035675
```

```
sha256=54dcf4e3c78a57ff57cd6d7851f1d36415c6defb12bb429274cae49d29234f1
```

```
f
```

```
  Stored in directory:
```

```
/root/.cache/pip/wheels/c9/dd/0f/a6a16f9f3b0236733d257b4b4ea91b548b984
a34led3b8f38c
```

```
Successfully built torch-sparse
```

```
Installing collected packages: torch-sparse
```

```
Successfully installed torch-sparse-0.6.18
```

```
Looking in links: https://data.pyg.org/whl/torch-.1.0+cu118+.html
```

```
Collecting torch-cluster
```

```
  Downloading torch_cluster-1.6.3.tar.gz (54 kB)
```

```
54.5/54.5 kB 2.4 MB/s eta
```

```
0:00:00
```

```
etadata (setup.py) ... ent already satisfied: scipy in
```

```
/usr/local/lib/python3.10/dist-packages (from torch-cluster) (1.11.3)
```

```
Requirement already satisfied: numpy<1.28.0,>=1.21.6 in
```

```
/usr/local/lib/python3.10/dist-packages (from scipy->torch-cluster)
```

```
(1.23.5)
```

```
Building wheels for collected packages: torch-cluster
```

```
  Building wheel for torch-cluster (setup.py) ... e=torch_cluster-
```

```
1.6.3-cp310-cp310-linux_x86_64.whl size=690648
```

```
sha256=283782d23d41f8b43a631cd720aaa14565291e0ba1437172cb1733924568387
```

```
6
```

```
  Stored in directory:
```

```
/root/.cache/pip/wheels/51/78/c3/536637b3cdcc3313aa5e8851a6c72b97f6a01
877e68c7595e3
```

```
Successfully built torch-cluster
```

```
Installing collected packages: torch-cluster
Successfully installed torch-cluster-1.6.3
Looking in links: https://data.pyg.org/whl/torch-.1.0+cu118+.html
Collecting torch-spline-conv
  Downloading torch_spline_conv-1.2.2.tar.gz (25 kB)
  Preparing metadata (setup.py) ... e=torch_spline_conv-1.2.2-cp310-
cp310-linux_x86_64.whl size=198031
sha256=99ef42bbcec69dd83ebfcab0e63979e417362a94fc24ef25b9e538c389eca7b
4
  Stored in directory:
  /root/.cache/pip/wheels/fc/34/be/187e4b5f5ccefecca2c1a5dfc8da244ec50ba
  a1f33c7b8c9a1
Successfully built torch-spline-conv
Installing collected packages: torch-spline-conv
Successfully installed torch-spline-conv-1.2.2
Collecting torch-geometric
  Downloading torch_geometric-2.4.0-py3-none-any.whl (1.0 MB)
  1.0/1.0 MB 9.4 MB/s eta
0:00:00
Requirement already satisfied: tqdm in /usr/local/lib/python3.10/dist-packages
(from torch-geometric) (4.66.1)
Requirement already satisfied: numpy in
/usr/local/lib/python3.10/dist-packages (from torch-geometric)
(1.23.5)
Requirement already satisfied: scipy in
/usr/local/lib/python3.10/dist-packages (from torch-geometric)
(1.11.3)
Requirement already satisfied: jinja2 in
/usr/local/lib/python3.10/dist-packages (from torch-geometric) (3.1.2)
Requirement already satisfied: requests in
/usr/local/lib/python3.10/dist-packages (from torch-geometric)
(2.31.0)
Requirement already satisfied: pyparsing in
/usr/local/lib/python3.10/dist-packages (from torch-geometric) (3.1.1)
Requirement already satisfied: scikit-learn in
/usr/local/lib/python3.10/dist-packages (from torch-geometric) (1.2.2)
Requirement already satisfied: psutil>=5.8.0 in
/usr/local/lib/python3.10/dist-packages (from torch-geometric) (5.9.5)
Requirement already satisfied: MarkupSafe>=2.0 in
/usr/local/lib/python3.10/dist-packages (from jinja2->torch-geometric)
(2.1.3)
Requirement already satisfied: charset-normalizer<4,>=2 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-
geometric) (3.3.2)
Requirement already satisfied: idna<4,>=2.5 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-
geometric) (3.4)
Requirement already satisfied: urllib3<3,>=1.21.1 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-
```

```
geometric) (2.0.7)
Requirement already satisfied: certifi>=2017.4.17 in
/usr/local/lib/python3.10/dist-packages (from requests->torch-
geometric) (2023.7.22)
Requirement already satisfied: joblib>=1.1.1 in
/usr/local/lib/python3.10/dist-packages (from scikit-learn->torch-
geometric) (1.3.2)
Requirement already satisfied: threadpoolctl>=2.0.0 in
/usr/local/lib/python3.10/dist-packages (from scikit-learn->torch-
geometric) (3.2.0)
Installing collected packages: torch-geometric
Successfully installed torch-geometric-2.4.0
```